

✓ Retail Store Sales – Exploratory Data Analysis

Dataset: 12,575 transactions · 11 columns · 1 Jan 2022 – 18 Jan 2025

Contents

1. Setup & Data Loading
2. Data Understanding
3. Data Cleaning
4. Feature Engineering
5. Outlier Detection & Treatment
6. Univariate Analysis
7. Time-Based Analysis
8. Category Analysis
9. Key Findings & Recommendations

✓ 1. Setup & Data Loading

```
#importing major libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
#reading the csv
df=pd.read_csv('/content/retail_store_sales.csv')
```

✓ 2. Data Understanding

✓ First Look at the Data

```
#inspecting the first few records
df.head(5)
```

	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10.0	185.0	
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9.0	261.0	
2	TXN_0202710	CUST_02	Butchers	Item_12_BUT	21.5	2.0	43.0	

```
#inspecting the shape of dataset
print(f'This dataset contains {df.shape[0]} rows and {df.shape[1]} columns.'
```

This dataset contains 12575 rows and 11 columns.

```
#checking basic structure and datatypes
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction ID         12575 non-null  str
1   Customer ID            12575 non-null  str
2   Category               12575 non-null  str
3   Item                   11362 non-null  str
4   Price Per Unit         11966 non-null  float64
5   Quantity               11971 non-null  float64
6   Total Spent            11971 non-null  float64
7   Payment Method         12575 non-null  str
8   Location               12575 non-null  str
9   Transaction Date       12575 non-null  str
10  Discount Applied       8376 non-null   object
dtypes: float64(3), object(1), str(7)
memory usage: 1.1+ MB
```

Insights

- The data has **12,575 transactions and 11 columns**: IDs, category/item, price, quantity, total, payment method, location, date and discount flag.
- `Transaction Date` is stored as text (`object`) and must be converted to a date type.
- `Item`, `Price Per Unit`, `Quantity`, `Total Spent` and `Discount Applied` have fewer non-null values than the other columns, so they contain missing data.

✓ Duplicates & Missing Values

```
#checking if there are any duplicate records
print(f'This dataset contains {df.duplicated().sum()} duplicate records.')
```

This dataset contains 0 duplicate records.

```
#inspecting missing records in a dataset
df.isna().sum()
```

```
Transaction ID      0
Customer ID         0
Category            0
Item               1213
Price Per Unit      609
Quantity           604
Total Spent         604
Payment Method      0
Location            0
Transaction Date    0
Discount Applied    4199
dtype: int64
```

Insights

- **No duplicate records**, so every row is a unique transaction.
- Missing values: `Item` 1,213 (9.6%) · `Price Per Unit` 609 · `Quantity` 604 · `Total Spent` 604 · `Discount Applied` 4,199 (33%).
- The missing values are not random: the 609 missing prices and the 604 missing quantities/totals each occur in rows where `Item` is *also* missing. This helps us repair them accurately (see 3.1).

Unique Values

```
#inspecting unique values for each column except transaction id, customer id
df_cols=set(df.columns)
exclude ={'Transaction ID','Customer ID','Transaction Date'}
df_cols=df_cols.difference(exclude)
for col in df_cols:
    print(col)
    print(df[col].unique())
```

```
Category
<StringArray>
[
    'Patisserie',
    'Butchers',
    'Food',
    'Electric household essentials', 'Computers and electric accessories']
Length: 8, dtype: str
Quantity
[10.  9.  2.  7.  8. nan  1.  3.  6.  4.  5.]
Location
<StringArray>
['Online', 'In-store']
```

```

Length: 2, dtype: str
Price Per Unit
[18.5 29.  21.5 27.5 12.5 nan  5.  33.5 36.5  8.   6.5 39.5 24.5 23.
 35. 14.   9.5 41.  20. 38.  15.5 11.  32.  26. 30.5 17. ]
Payment Method
<StringArray>
['Digital Wallet', 'Credit Card', 'Cash']
Length: 3, dtype: str
Total Spent
[185. 261.  43.  247.5  87.5 200.  40.   nan  27.5 109.5  72.  52.
 45.5 237.  55.  232.  275.  23.  126.  105.  66.5  18.5  49.  134.
 410. 245. 182.5 100.  196. 315. 287.  76.  92.5 190. 255.5 276.5
 46.5  8.  107.5 165.  80. 192.5 335.  66. 215.  96.  42.  11.
 234. 316.  82.5 180. 365.  39. 172. 122.  30.  84. 320. 219.
 67. 146. 290.  70. 160.  82.  14. 355.5 124.  28.5 47.5 193.5
 38. 12.5 140. 120. 183. 305.  41. 155.  19.5 33. 108.5 119.
 280. 62.  32. 380. 304. 139.5 114. 192. 167.5 88. 395. 158.
 25. 50.  32.5 15.  33.5 24. 111.  46.  36.5 62.5 161.  26.
 98. 85. 228.  91.5 93.  35. 208. 100.5  9.5 10. 138. 288.
 60. 123.  73. 197.5 69.  61. 116.  55.5  6.5 152.5 118.5  74.
 205. 150.5 110.  19.  48. 175. 369.  37.5 65. 170.  77.5 129.5
 15.5 24.5 147.  21.5 31.  99. 128.  58.5 266.  29. 268.  64.
 115. 64.5 260. 207. 224. 328.  92. 244.  39.5 16. 328.5 182.
 44. 129. 301.5 57.   5. 122.5 220. 112.5 68. 292.  28. 342.
 102. 77. 148. 125. 210. 137.5 95. 136. 145. 153.  79.  78.
 51. 256. 152. 230. 274.5 184. 171.5 20.  58. 213.5 203. 104.
 201. 234.5 87.  56.  86.  30.5 85.5 174.  22.  34. 156. 350.
 75. 246. 220.5 166.5 45. 112. 164. 130.  37.  73.5 13.  17. ]
Item
<StringArray>
[ 'Item_10_PAT', 'Item_17_MILK', 'Item_12_BUT', 'Item_16_BEV',
  'Item_6_FOOD',          nan, 'Item_1_FOOD', 'Item_16_FUR',
  'Item_22_BUT', 'Item_3_BUT',
  ...
  'Item_3_PAT', 'Item_13_FUR', 'Item_18_PAT', 'Item_12_MILK',
  'Item_10_MILK', 'Item_16_FOOD', 'Item_19_FOOD', 'Item_14_EHE',
  'Item_3_EHE', 'Item_9_BUT']
Length: 201, dtype: str
Discount Applied
[True False nan]

```

Insights

- 8 categories, 3 payment methods (Cash, Credit Card, Digital Wallet) and 2 locations (Online, In-store).
- `Discount Applied` has three states: `True`, `False` and `NaN` (unknown).
- Item names end with a **category code** (e.g. `Item_10_PAT` = Patisserie, `_BEV` = Beverages), which tells us each item belongs to exactly one category.

✓ 3. Data Cleaning

Before filling anything with a mean or mode, we test whether the data follows hidden rules.

```
#checking hidden rules in the data
known = df.dropna(subset=['Item','Price Per Unit'])
chk = df.dropna(subset=['Price Per Unit','Quantity','Total Spent'])
print('Rows where Total Spent != Price x Quantity      : ', (abs(chk['Total S
print('Items having more than one price                : ', (known.groupby('I
print('(Category, Price) pairs that match >1 item       : ', (known.groupby(['
```

```
Rows where Total Spent != Price x Quantity      : 0
Items having more than one price                : 0
(Category, Price) pairs that match >1 item       : 0
```

Insights

- $\text{Total Spent} = \text{Price Per Unit} \times \text{Quantity}$ holds in **every** complete row.
- **Each item has exactly one price**, and a $(\text{Category}, \text{Price})$ pair points to exactly one item.
- Therefore a missing price can be recovered from $\text{Total} \div \text{Quantity}$, and a missing item from $\text{Category} + \text{Price}$ – no guessing needed.

✓ Price Per Unit & Item

```
#recovering missing price per unit from total spent / quantity
df['Price Per Unit']=df['Price Per Unit'].fillna(df['Total Spent']/df['Quant
```

```
#filling item column using category + price
known = df.dropna(subset=['Item','Price Per Unit'])
lookup = known.drop_duplicates(['Category','Price Per Unit']).set_index(['Ca
keys = pd.MultiIndex.from_frame(df[['Category','Price Per Unit']])
df['Item'] = df['Item'].fillna(pd.Series(lookup.reindex(keys).values, index=
```

```
#checking distinct prices for each item category
df.groupby('Item')['Price Per Unit'].value_counts()
```

Item	Price Per Unit	
Item_10_BEV	18.5	23
Item_10_BUT	18.5	16
Item_10_CEA	18.5	79
Item_10_EHE	18.5	45
Item_10_FOOD	18.5	60
		..
Item_9_EHE	17.0	63
Item_9_FOOD	17.0	78
Item_9_FUR	17.0	16
Item_9_MILK	17.0	11
Item_9_PAT	17.0	59

Name: count, Length: 200, dtype: int64

```
#filling missing price per unit with most frequent mode for each item catego
df['Price Per Unit']=df.groupby('Item')['Price Per Unit'].transform(lambda x
```

Insights

- All **1,213 missing items** and **609 missing prices** were recovered exactly from the data's own rules.
- Every item now shows a single price, confirming the repair is consistent. The last cell is kept from the original – it is now a safety net that changes nothing.

✓ Quantity & Total Spent

```
#filling quantity column with median values
df['Qty_imputed']=df['Quantity'].isna()
df['Quantity']=df['Quantity'].fillna(df['Quantity'].median())
```

```
#filling total spent with price*quantity
df['Total Spent']=df['Quantity']*df['Price Per Unit']
```

✓ Discount Applied

```
#mapping true with yes, false with no and nan with unknown
df['Discount Applied']=df['Discount Applied'].map({True: 'Yes',False: 'No'})
```

Insights

- Only the **604 rows (4.8%)** with both Quantity and Total Spent missing needed an estimate – everything else was calculated exactly.
- Missing discount status (33%) is kept as its own **Unknown** group instead of guessing Yes/No.

✓ Final Check & Data Types

```
#rechecking null values in the dataframe
df.isna().sum()
```

```
Transaction ID      0
Customer ID         0
Category            0
Item                0
Price Per Unit      0
Quantity            0
Total Spent         0
Payment Method      0
Location            0
Transaction Date    0
Discount Applied    0
```

```
Qty_imputed      0
dtype: int64
```

```
#changing datatypes
df['Quantity']=df['Quantity'].astype('int64')
df['Transaction Date']=pd.to_datetime(df['Transaction Date'])
```

```
#rechecking datatypes
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction ID         12575 non-null  str
1   Customer ID           12575 non-null  str
2   Category              12575 non-null  str
3   Item                  12575 non-null  str
4   Price Per Unit        12575 non-null  float64
5   Quantity              12575 non-null  int64
6   Total Spent           12575 non-null  float64
7   Payment Method        12575 non-null  str
8   Location              12575 non-null  str
9   Transaction Date      12575 non-null  datetime64[us]
10  Discount Applied      12575 non-null  str
11  Qty_imputed           12575 non-null  bool
dtypes: bool(1), datetime64[us](1), float64(2), int64(1), str(7)
memory usage: 1.1 MB
```

Insights

- **No missing values remain** and all columns now have suitable types (`Quantity` as integer, `Transaction Date` as datetime).

✓ 4. Feature Engineering

```
#Date Features
df['Year'] = df['Transaction Date'].dt.year
df['Quarter'] = df['Transaction Date'].dt.quarter
df['Month'] = df['Transaction Date'].dt.month
df['Month_name'] = df['Transaction Date'].dt.month_name()
df['Day_of_week'] = df['Transaction Date'].dt.day_name()
df['is_weekend'] = df['Transaction Date'].dt.dayofweek >= 5
df['Year_month'] = df['Transaction Date'].dt.to_period('M').dt.to_timestamp()
print(df['Transaction Date'].min().date(), 'to', df['Transaction Date'].max().date())
df.dtypes
```

```
2022-01-01 to 2025-01-18
Transaction ID      str
Customer ID         str
Category            str
Item                str
```

```

Price Per Unit          float64
Quantity                int64
Total Spent             float64
Payment Method          str
Location                str
Transaction Date        datetime64[us]
Discount Applied        str
Qty_imputed             bool
Year                   int32
Quarter                int32
Month                  int32
Month_name              str
Day_of_week            str
is_weekend              bool
Year_month              datetime64[us]
dtype: object

```

Insights

- The data covers **1 Jan 2022 to 18 Jan 2025**.
- ⚠ Because it ends on the 18th, **January 2025 is a partial month**. It is excluded from the monthly trend and month × year heatmap below, otherwise it looks like a false "revenue crash".

```

#setting color palette
sns.set_palette('Blues_d')

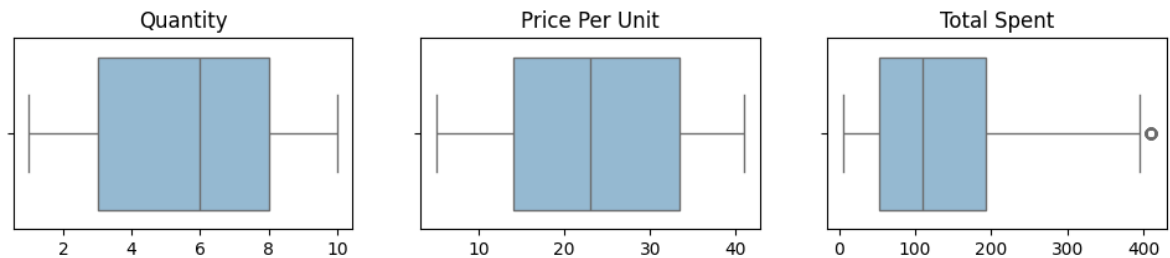
```

✓ 5. Outlier Detection & Treatment

```

#outlier detection using boxplots
fig,axs=plt.subplots(nrows=1,ncols=3,figsize=(12,2))
sns.boxplot(data=df,x='Quantity',ax =axs[0])
axs[0].set_title('Quantity')
axs[0].set_xlabel('')
sns.boxplot(data=df,x='Price Per Unit',ax =axs[1])
axs[1].set_title('Price Per Unit')
axs[1].set_xlabel('')
sns.boxplot(data=df,x='Total Spent',ax =axs[2])
axs[2].set_title('Total Spent')
axs[2].set_xlabel('')
plt.show()

```



```
#outliers treatment
cols=['Quantity','Price Per Unit','Total Spent']
for col in cols:
    q1=df[col].quantile(0.25)
    q3=df[col].quantile(0.75)
    iqr=q3-q1
    lower_bound=q1-(1.5*iqr)
    upper_bound=q3+(1.5*iqr)
    df[col]=np.where(df[col]>upper_bound,upper_bound,df[col])
    df[col]=np.where(df[col]<lower_bound,lower_bound,df[col])
```

```
fig,axs=plt.subplots(nrows=1,ncols=3,figsize=(12,2))
sns.boxplot(data=df,x='Quantity',ax =axs[0])
axs[0].set_title('Quantity')
axs[0].set_xlabel('')
sns.boxplot(data=df,x='Price Per Unit',ax =axs[1])
axs[1].set_title('Price Per Unit')
axs[1].set_xlabel('')
sns.boxplot(data=df,x='Total Spent',ax =axs[2])
axs[2].set_title('Total Spent')
axs[2].set_xlabel('')
plt.show()
```



Insights

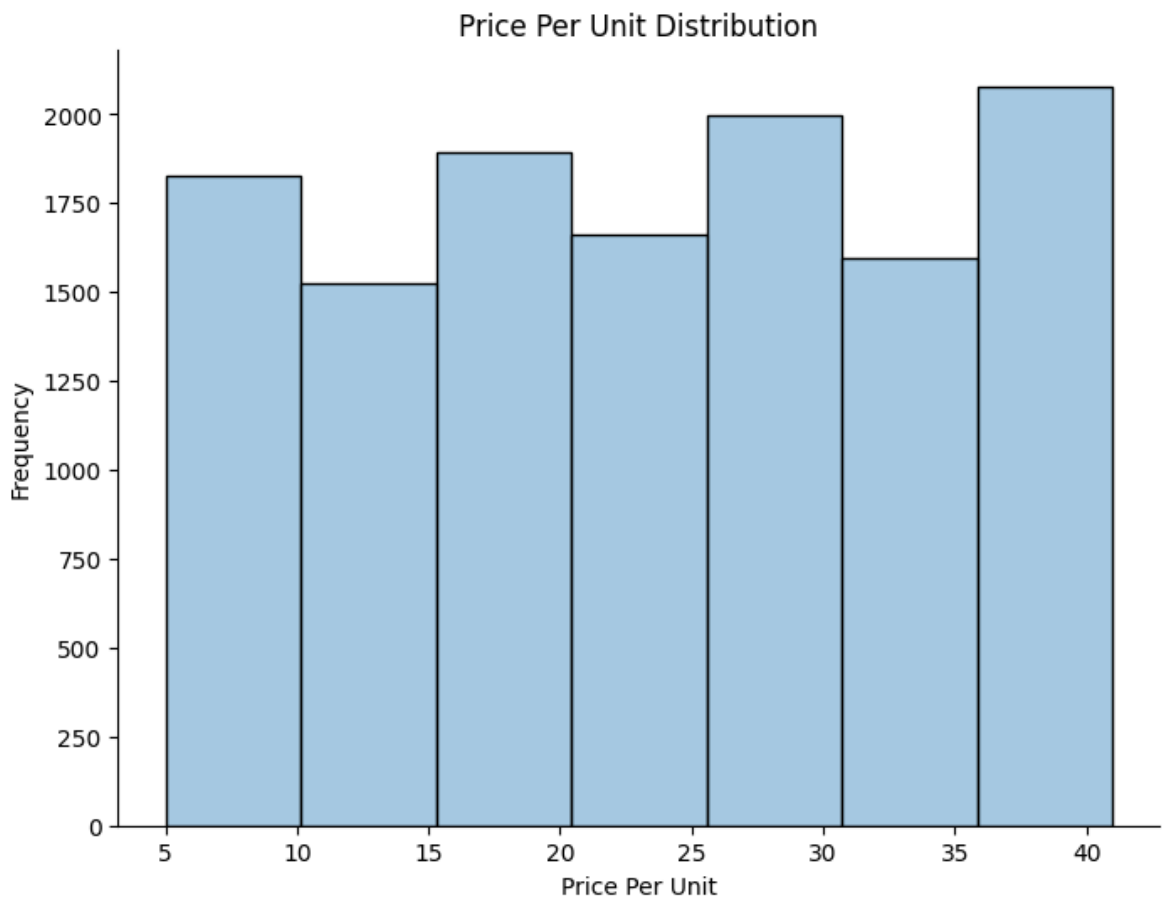
- **Quantity** (1–10) and **Price Per Unit** (5–41) have **no outliers** – they sit in fixed business ranges.

- `Total Spent` has **60 baskets (0.5%) between about 403 and 410** flagged by the IQR rule. These are **valid values** (the largest possible basket is $41 \times 10 = 410$), so capping them at ≈ 402 changes total revenue by only about 0.03%.
- The boxplots before and after look almost identical, which confirms the data is clean.

✓ 6. Univariate Analysis

✓ Price Per Unit

```
#price per unit distribution
plt.figure(figsize=(8,6))
sns.histplot(data=df,x='Price Per Unit',binwidth=5)
plt.title('Price Per Unit Distribution')
plt.xlabel('Price Per Unit')
plt.ylabel('Frequency')
plt.gca().spines[['top','right']].set_visible(False)
plt.show()
```

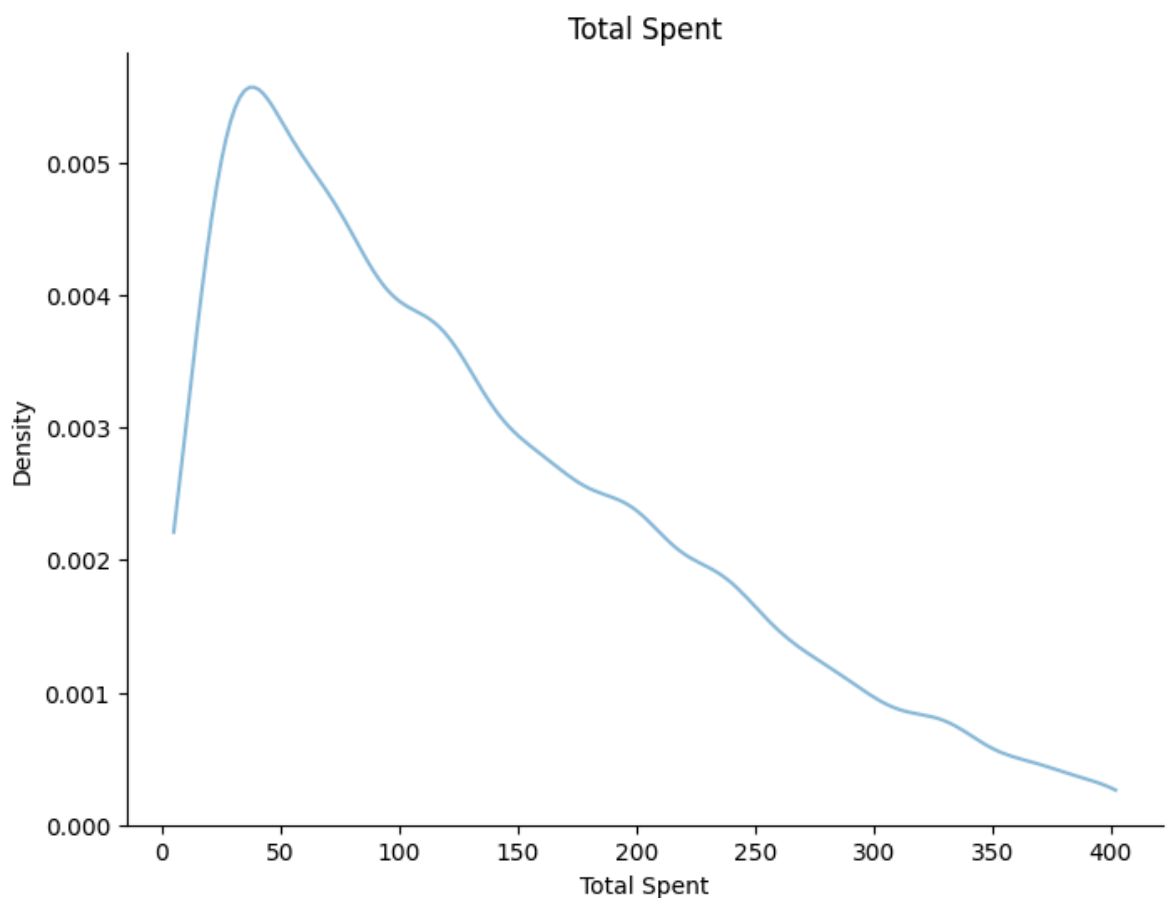


Insights

- Prices are spread **fairly evenly from 5 to 41** with no dominant price band. The store sells a balanced mix of cheap, mid-range and premium items.

✓ Total Spent

```
#total spent
plt.figure(figsize=(8,6))
sns.kdeplot(data=df,x='Total Spent',cut=0)
plt.title('Total Spent')
plt.gca().spines[['top','right']].set_visible(False)
plt.show()
```

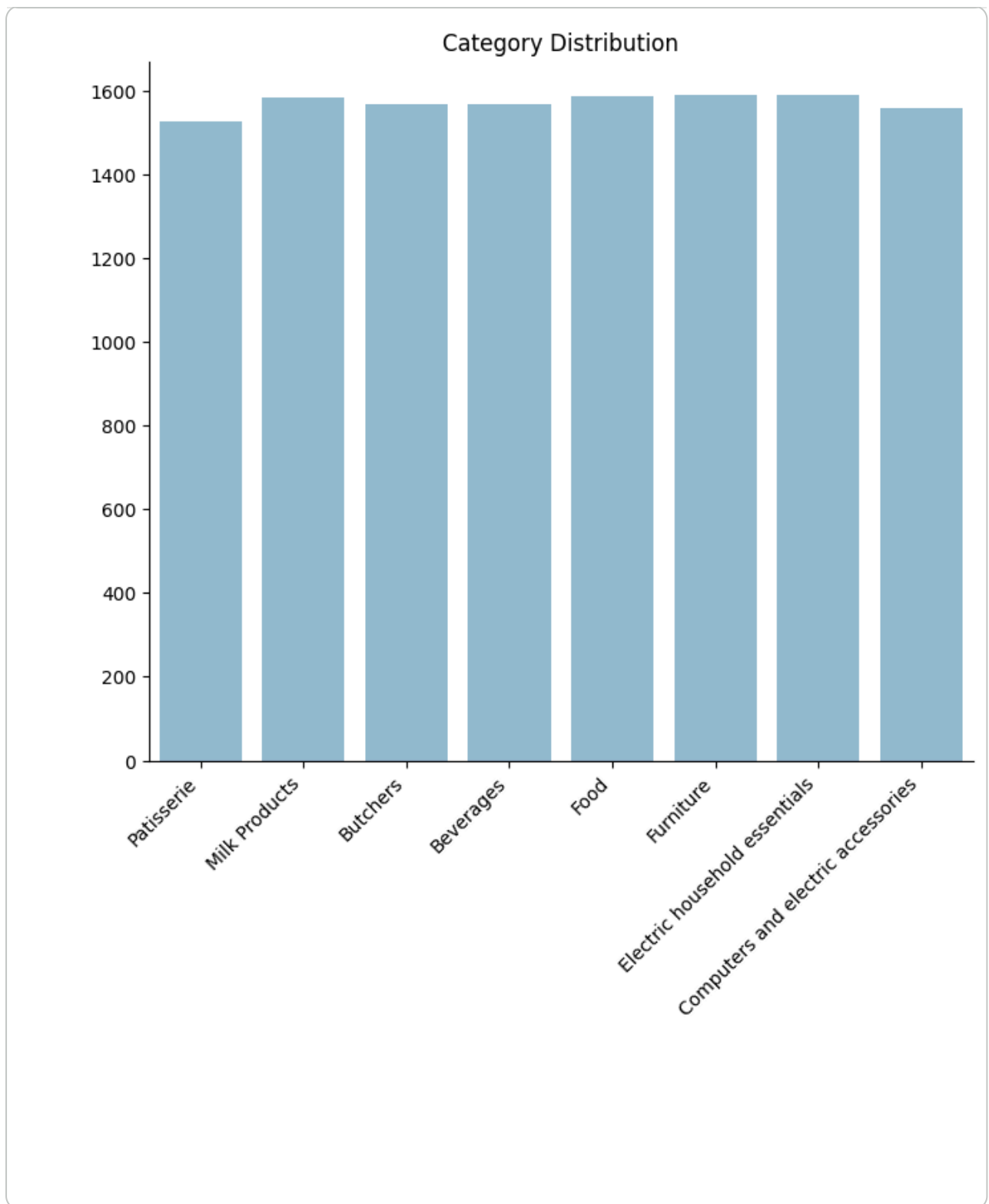


Insights

- The distribution is **right-skewed**: most transactions are small, with a long tail of larger baskets up to about 400.
- The **median (≈ 110)** is lower than the mean (≈ 130), so the median describes a "typical" transaction better.

✓ 6.3 Category Distribution

```
#category distribution
plt.figure(figsize=(8,6))
sns.countplot(data=df,x='Category')
plt.tight_layout()
plt.title('Category Distribution')
plt.xlabel('')
plt.ylabel('')
plt.gca().spines[['top','right']].set_visible(False)
plt.xticks(rotation=45, ha='right')
plt.show()
```



Insights

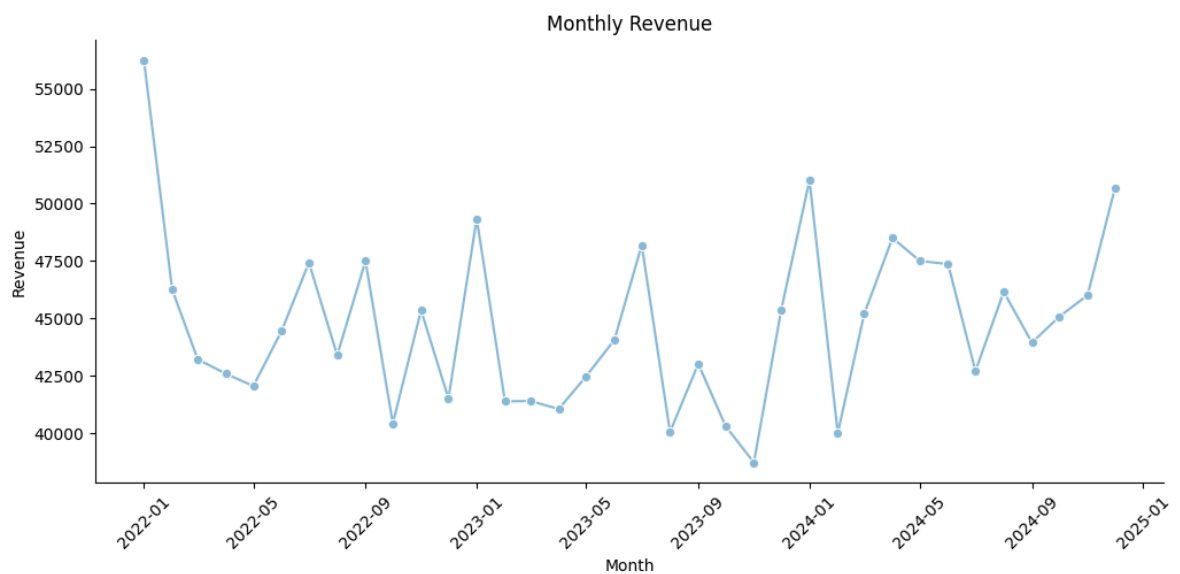
- All 8 categories have **almost the same number of transactions (about 1,530–1,590 each)**. No category wins on footfall, so differences in revenue come from *basket value*, not popularity.

✓ 7. Time-Based Analysis

✓ Monthly Revenue

```
#monthly revenue
monthly = df[df['Year_month'] < '2025-01-01'].groupby('Year_month')['Total S

plt.figure(figsize=(12,5))
sns.lineplot(data=monthly, x='Year_month', y='Total Spent', marker='o')
plt.title('Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.xticks(rotation=45)
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show()
```



Insights

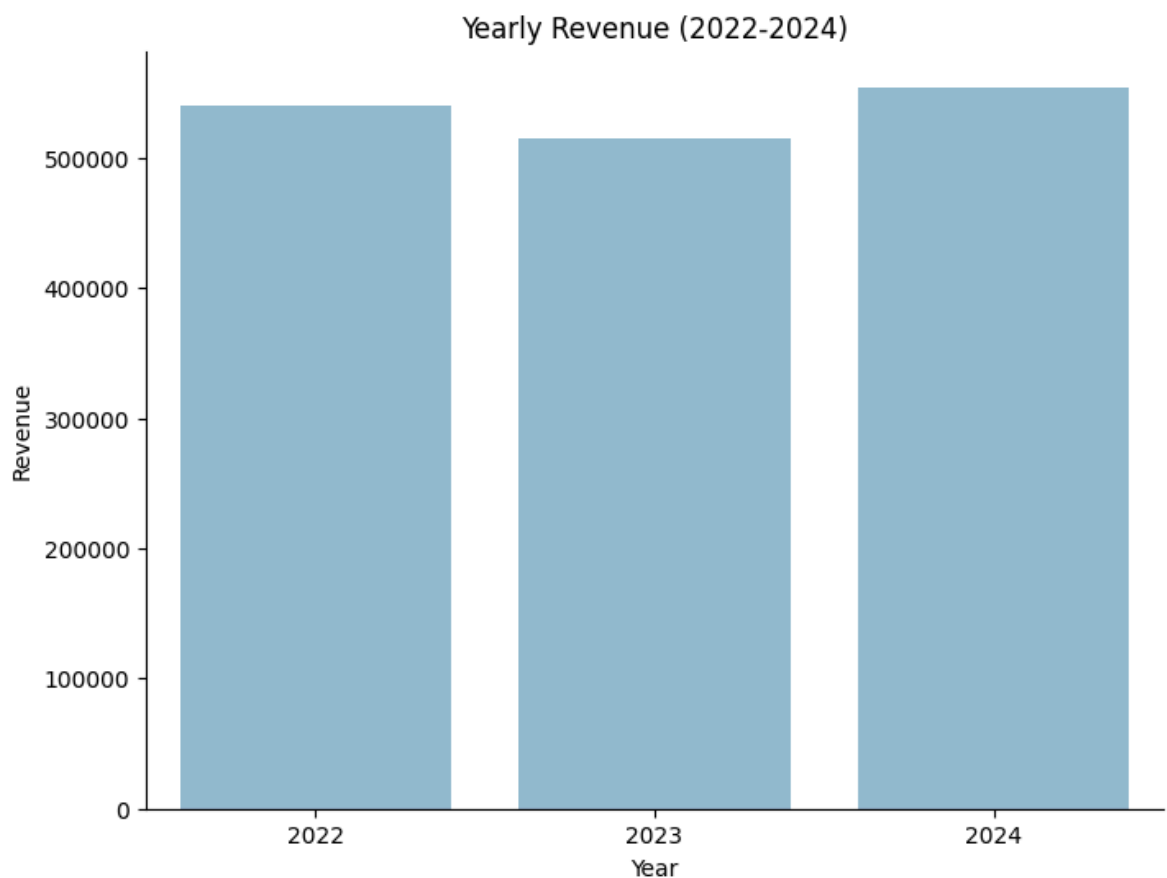
- Monthly revenue stays in a stable band of roughly **38k–56k** with **no strong long-term trend**.
- **January is the peak in every year**, and Jan 2022 is the highest month on record.
- The sharp fall seen in the original chart was **not real**: the data stops on 18 Jan 2025. Scaled to a full month, Jan 2025 would be about 46k, in line with earlier

Januaries.

✓ Yearly Revenue

```
#yearly revenue
yearly = df[df['Year'] < 2025].groupby('Year')['Total Spent'].sum().reset_index()

plt.figure(figsize=(8,6))
sns.barplot(data=yearly, x='Year', y='Total Spent')
plt.title('Yearly Revenue (2022-2024)')
plt.xlabel('Year')
plt.ylabel('Revenue')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show()
```



Insights

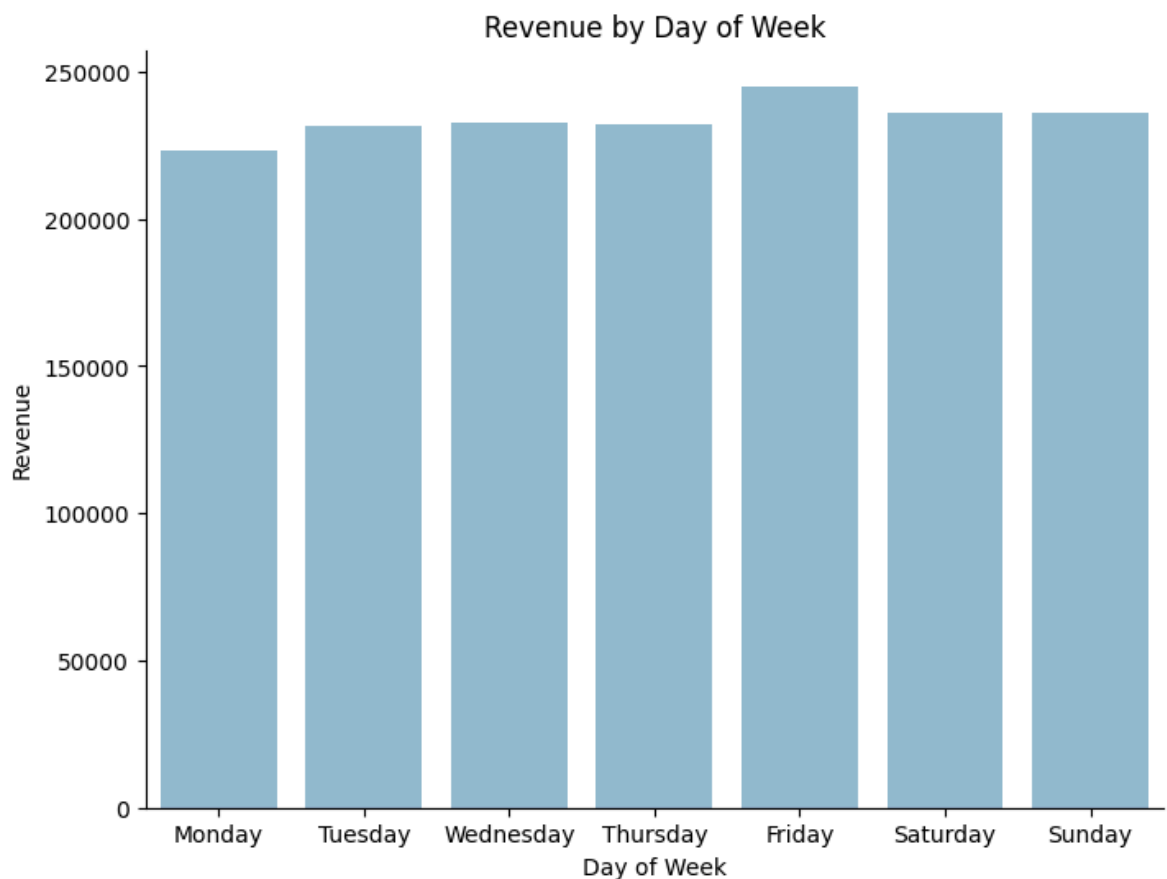
- Revenue went **2022 ≈ 540k → 2023 ≈ 515k (about -4.6%) → 2024 ≈ 554k (about +7.5%)**. 2024 is the best year, recovering from a dip in 2023.
- Changes follow the **number of transactions** (4,134 → 3,987 → 4,241), not basket size, which stays near 130 each year. Growth here is driven by customer

traffic, not higher prices.

✓ Revenue by Day of Week

```
#revenue by day of week
dow_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
dow = df.groupby('Day_of_week')['Total Spent'].sum().reindex(dow_order).reset_index()

plt.figure(figsize=(8,6))
sns.barplot(data=dow, x='Day_of_week', y='Total Spent')
plt.title('Revenue by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Revenue')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show()
```



Insights

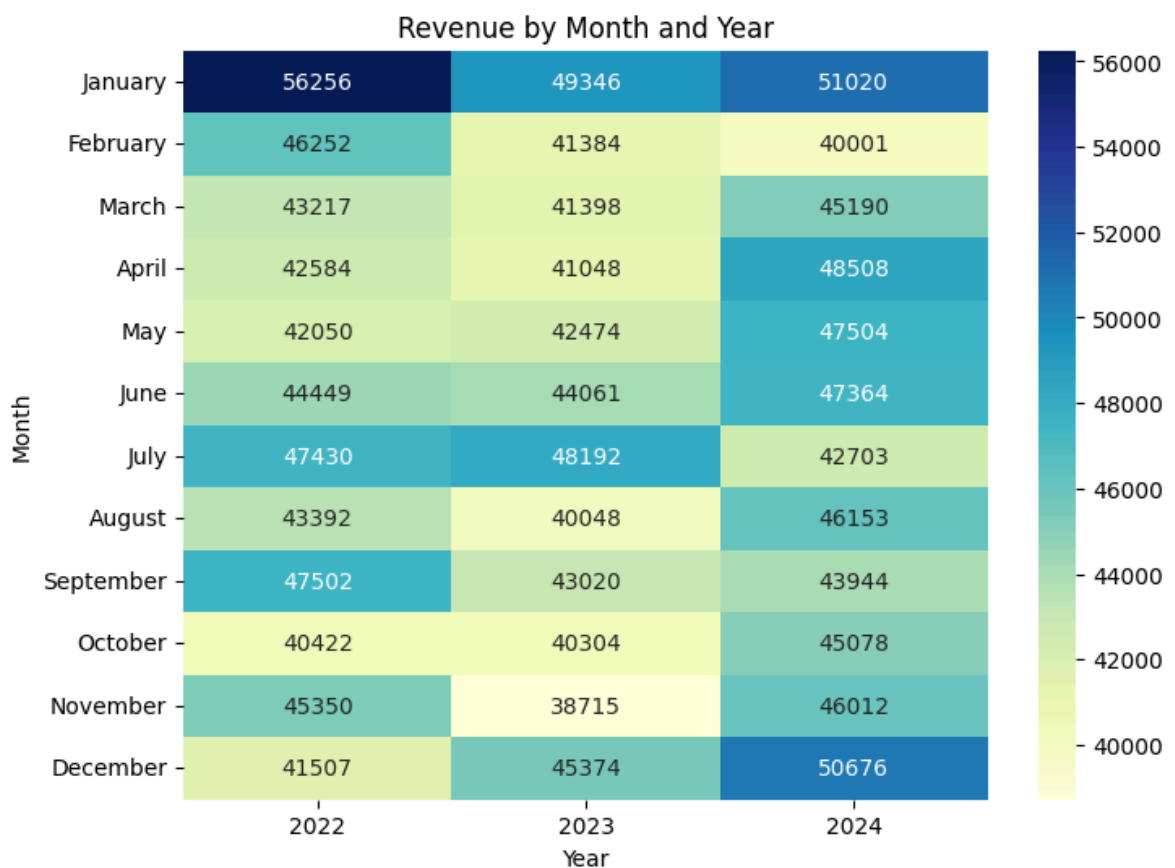
- **Friday is the strongest day (≈ 245k) and Monday the weakest (≈ 223k)** – a gap of about 10%.

- The other days are close together ($\approx 230\text{--}236\text{k}$). Weekends are *not* special: they hold about 29% of transactions, almost exactly their 2-in-7 share of days.

✓ Revenue by Month and Year

```
#visualising revenue by month and year through heatmap
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'Aug
pt = df[df['Year'] < 2025].pivot_table(index='Month_name', columns='Year', v

plt.figure(figsize=(8,6))
sns.heatmap(data=pt, annot=True, fmt='.0f', cmap='YlGnBu')
plt.title('Revenue by Month and Year')
plt.xlabel('Year')
plt.ylabel('Month')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show()
```



Insights

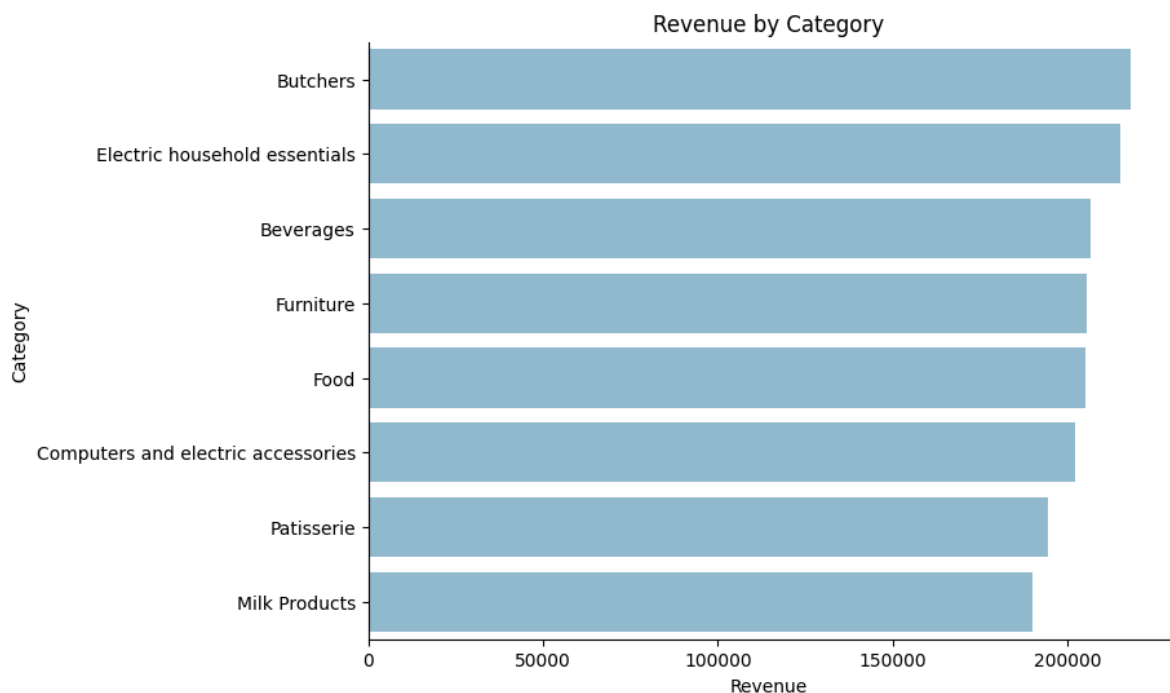
- **January is the strongest month in all three years** ($\approx 56\text{k}$, 49k and 51k) – the only month that is consistently on top.

- Over the three years combined, **October and February are the weakest months**, with July and December next-best after January.
- Apart from January, the pattern is **not very stable year to year**: July was strong in 2022–23 but weaker in 2024, while October and December were much stronger in 2024. Treat month-level differences other than January with caution.

✓ 8. Category Analysis

```
#revenue by category
cat_rev = df.groupby('Category')['Total Spent'].sum().sort_values(ascending=

plt.figure(figsize=(8,6))
sns.barplot(data=cat_rev, y='Category', x='Total Spent')
plt.title('Revenue by Category')
plt.xlabel('Revenue')
plt.ylabel('Category')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show()
```



```
#revenue share by category
```

```
cat_rev['Share_%'] = (cat_rev['Total Spent'] / cat_rev['Total Spent'].sum())
```

```
cat_rev
```

	Category	Total Spent	Share_%
0	Butchers	218065.0	13.3
1	Electric household essentials	215217.5	13.1
2	Beverages	206766.5	12.6
3	Furniture	205326.0	12.5
4	Food	205129.0	12.5
5	Computers and electric accessories	201000.5	12.2